

Uni-Thrift

A Peer-to-Peer Academic Resource Exchange

Fundamentals of Software Engineering

Date of Submission: May 3, 2026

Group Members:

- **Muhammad Soban Sohail (Roll Number: 24L-2545)** - [Lead & Database Architect](#)
 - **Saad Jahangir (Roll Number: 24L-2516)** - [Full-Stack Developer](#)
 - **Mahad Jawad Rana (Roll Number: 24L-2504)** - [Requirement Engineer & QA](#)
-

Project Introduction

Project Title: UniThrift: A Peer-to-Peer Academic Resource Exchange and Emergency Borrowing System

Introduction: UniThrift is a full-stack, hybrid campus-exclusive web application designed to gamify and streamline the buying, selling, and borrowing of academic resources among students. The platform was developed incrementally across three Agile sprints:

- **Sprint 1 (User Management & Foundation):** Established the core SQL database, JWT-based secure authentication restricted strictly to nu.edu.pk emails, and the foundational user profile management system.
- **Sprint 2 (Marketplace & Discovery):** Transformed the platform into a functional economy by introducing Item Uploads (physical and digital), an interactive Notice Board with departmental filtering, and a secure Wishlist feature.
- **Sprint 3 (Logistics & Accountability):** Solidified the community with a real-time S.O.S Emergency Request feed, quest-isolated In-App Messaging for handoff coordination, a Trust Rating System, and a Secure Digital Vault bypass for instant e-book access.

User Stories (All Sprints)

Story ID	Title	Priority	Description	Acceptance Criteria
US-01	University Email Sign-up	Critical	As a student, I want to sign up using my nu.edu.pk email to ensure the marketplace is exclusive to my peers.	Domain regex validation; bcrypt password hashing; Protected routes via JWT.
US-02	Profile Management	High	As a user, I want a profile page to track my active listings and view my total sold/borrowed statistics.	Dynamically calculates SQL aggregates based on transaction_id and item_id.
US-03	Item Uploads & Listings	High	As a seller, I want to upload items with cover images or secure PDFs to list them on the Notice Board.	Form validation; Handles multipart/form-data; Digital items default to 1 qty.
US-04	Departmental Filtering	Medium	As a user, I want to filter the Notice Board by department to easily find relevant academic supplies.	Feed auto-refreshes; Hides out-of-stock physical items but displays digital items.
US-05	Wishlist Implementation	Medium	As a browser, I want to toggle a heart icon to save listings to a private wishlist for later viewing.	Toggles visual state; Saves to DB; Protects route via authenticateToken middleware.
US-06	SOS Borrowing Requests	Critical	As a student, I want to broadcast a real-time SOS request to the feed for immediate peer assistance.	Updates feed in real-time; Cancels competing offers when a handshake is accepted.
US-07	In-App Messaging	High	As a user, I want to chat directly with my transaction partner to coordinate item handoffs safely.	Isolated by item_id & Users; Asynchronous polling; Parses UTC to local time.
US-08	Trust Rating System	Medium	As a user, I want to rate peers after a transaction to build global reliability scores.	Ranks 1-5 stars; Safely updates User's aggregate average via AVG() query.
US-09	Digital Vault Bypass	High	As a borrower of an E-book, I want instant access without waiting for physical handshakes.	Controller identifies Is_digital; Bypasses physical stored procedure; Streams PDF.

Design

Use Case Diagram

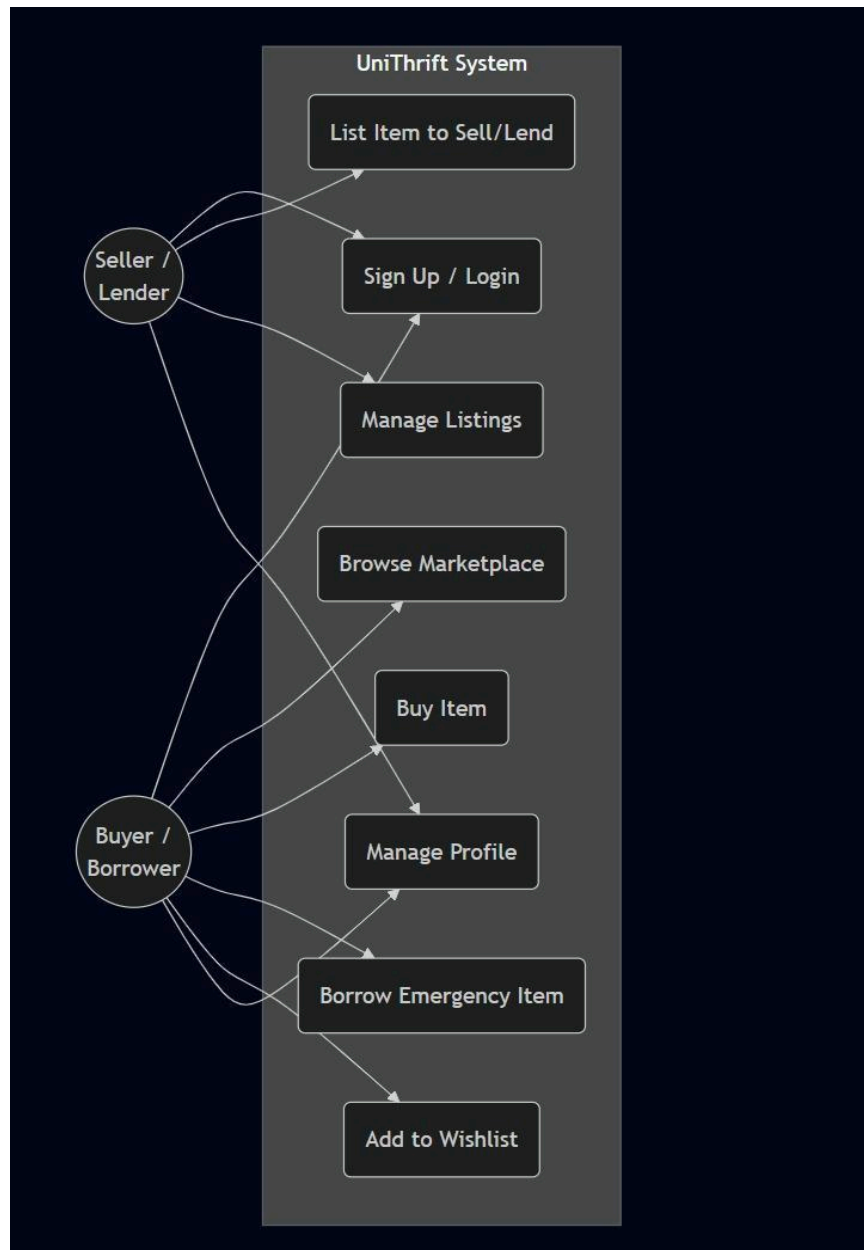


Figure 1: UniThrift Use Case Diagram — Seller/Lender and Buyer/Borrower actors

Entity Relationship Diagram (ERD)

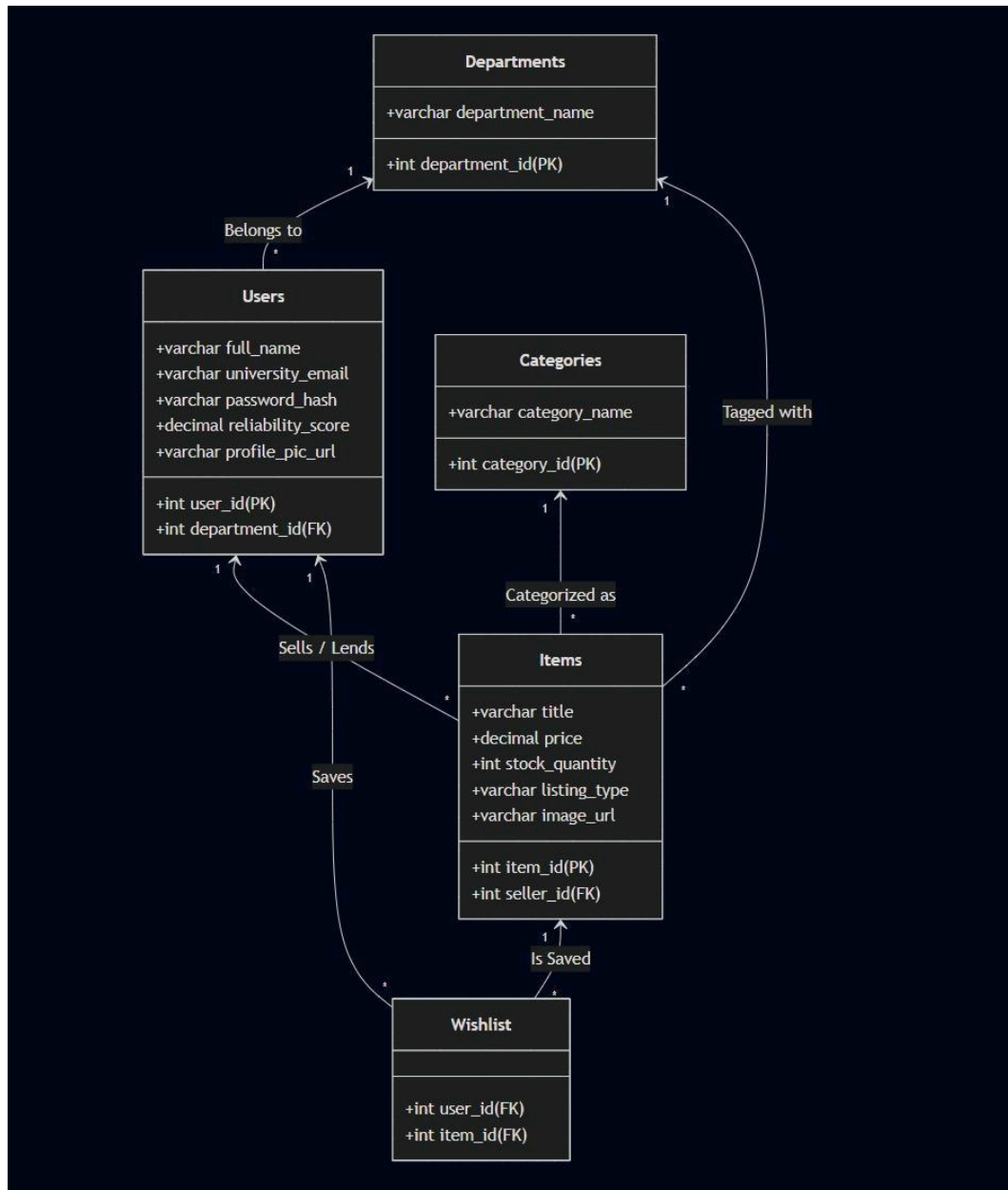


Figure 2: Database ERD — Departments, Users, Items, Categories, Wishlist

Flow Charts

FC-01: Buy Item Flow

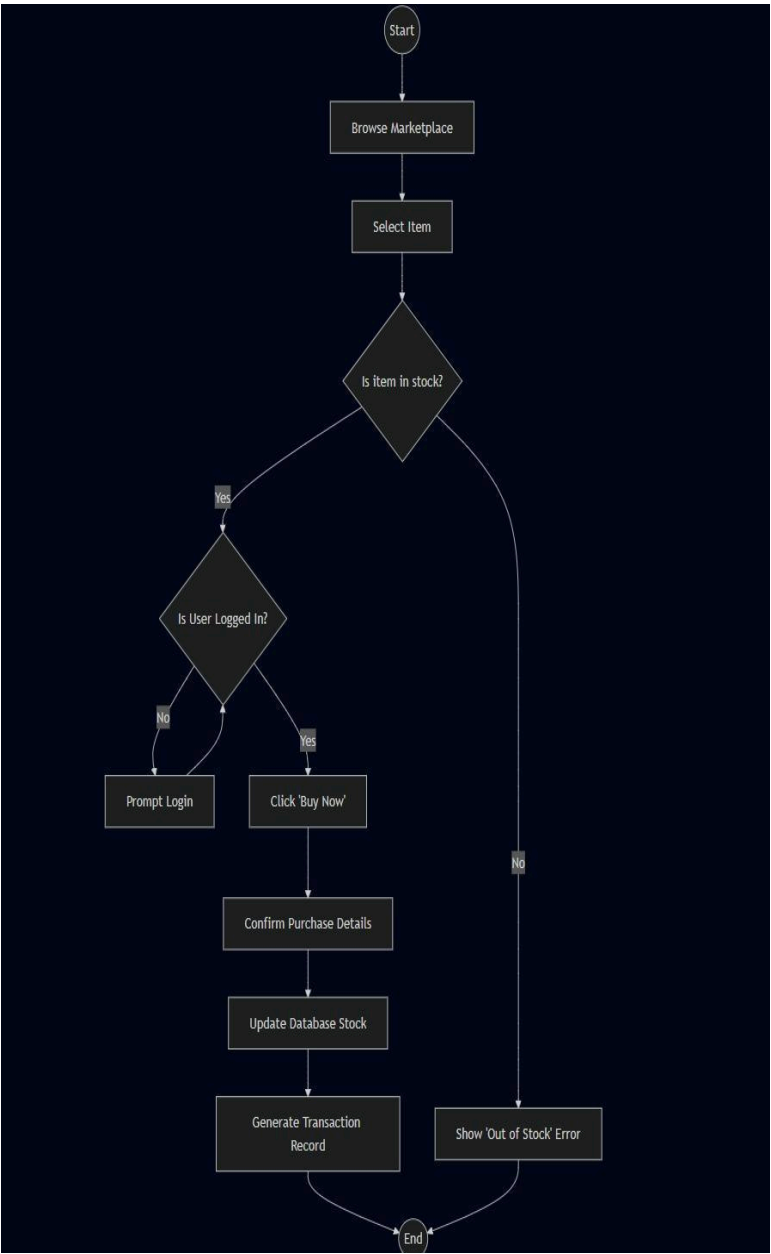


Figure 3: FC-01 — Buy Item Flowchart

FC-02: Add Listing Flow

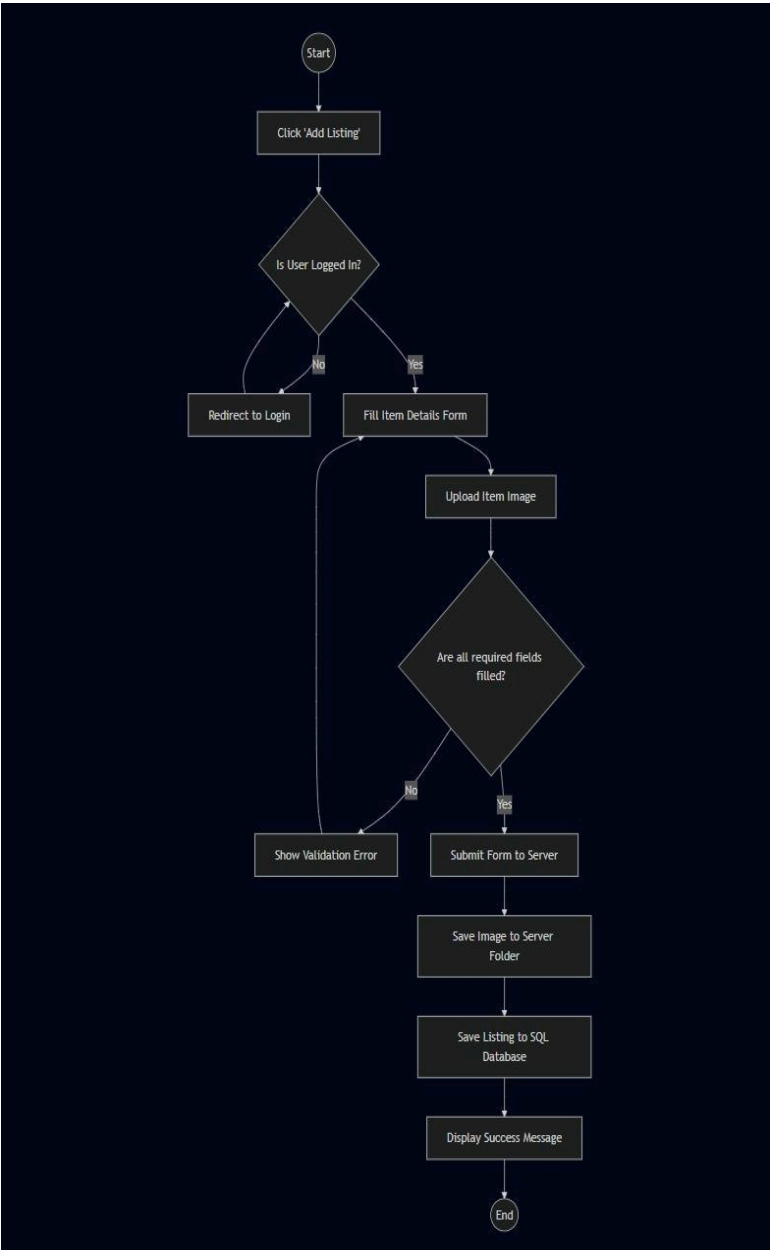


Figure 4: FC-02 — Add Listing Flowchart

FC-03: Borrow Item Flow

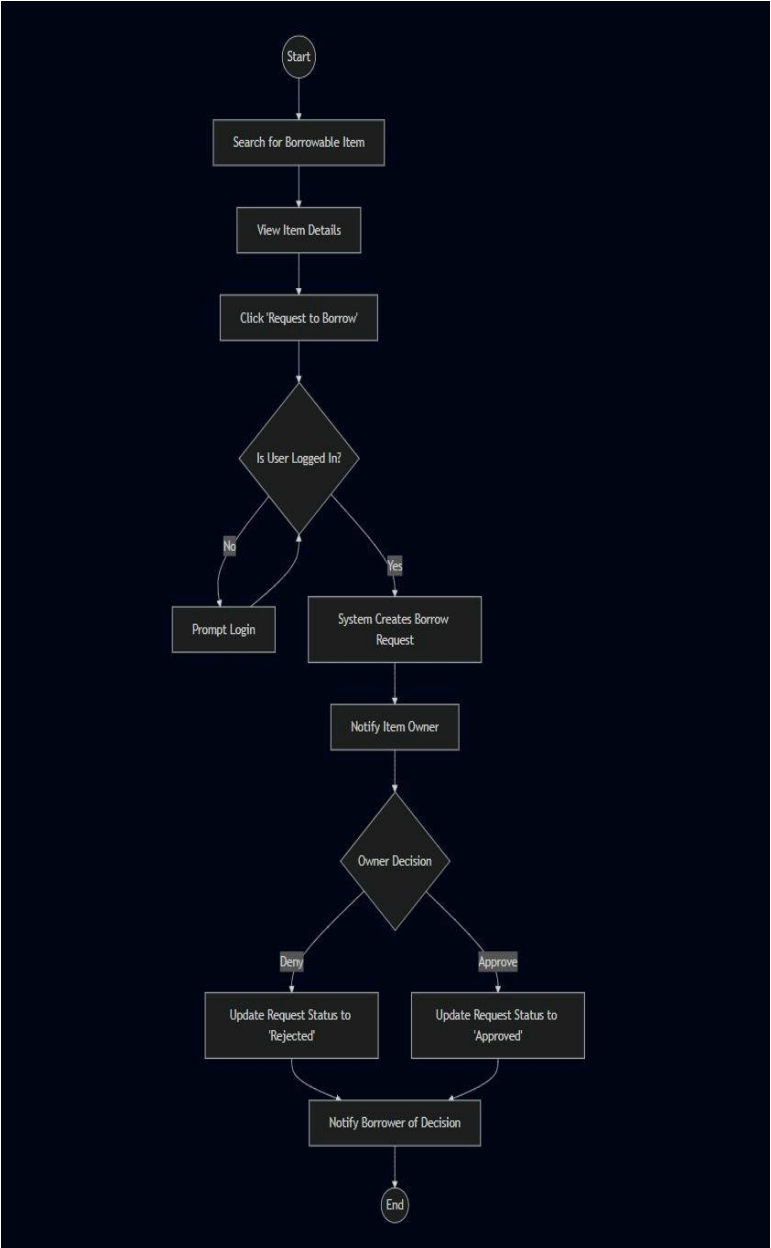


Figure 5: FC-03 — Borrow Item Flowchart

Sequence Diagrams

SD-01: Buy Item Sequence

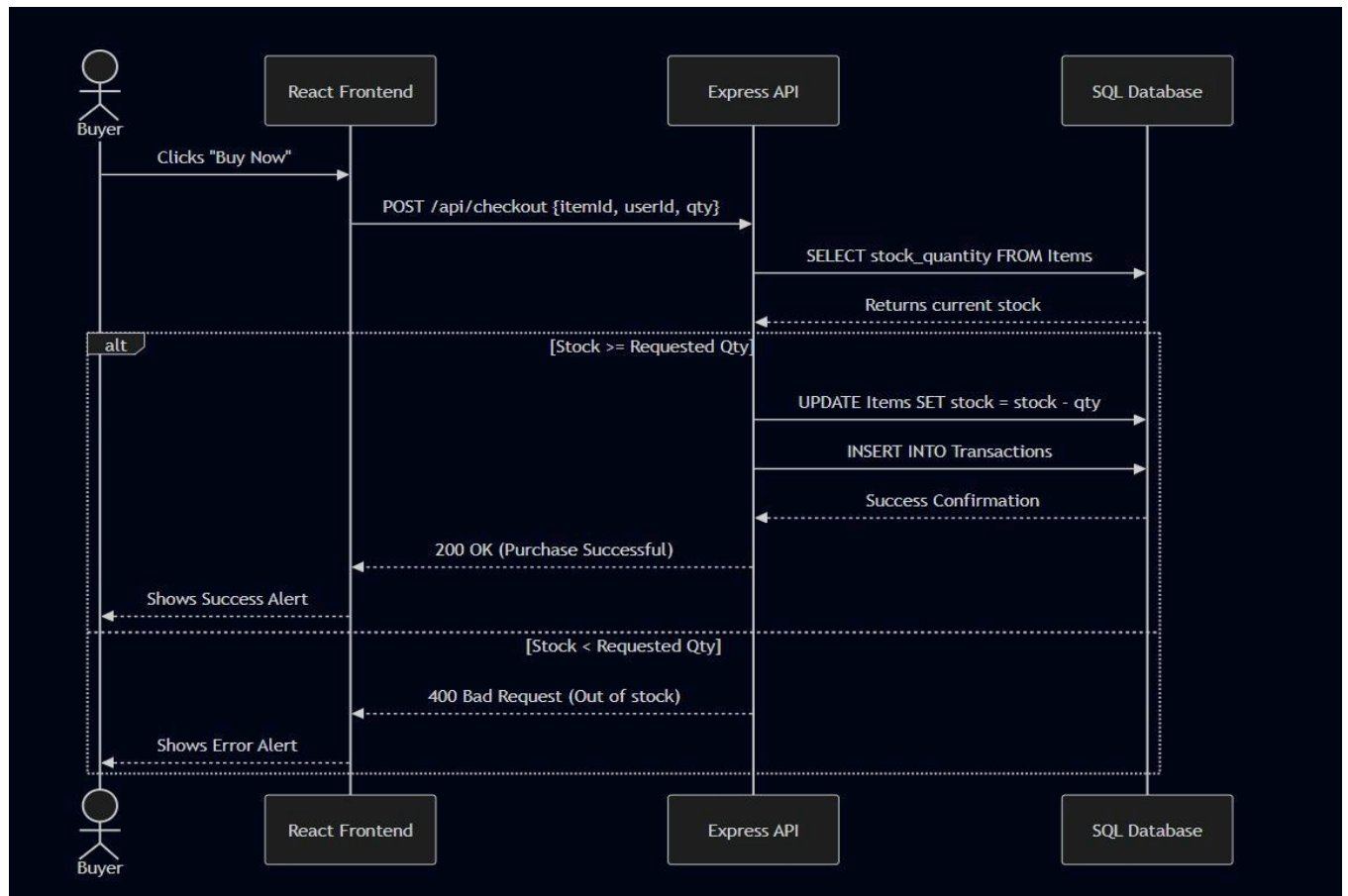


Figure 6: SD-01 — Buy Item Sequence Diagram (Buyer → React → Express → SQL)

SD-02: Add Listing Sequence

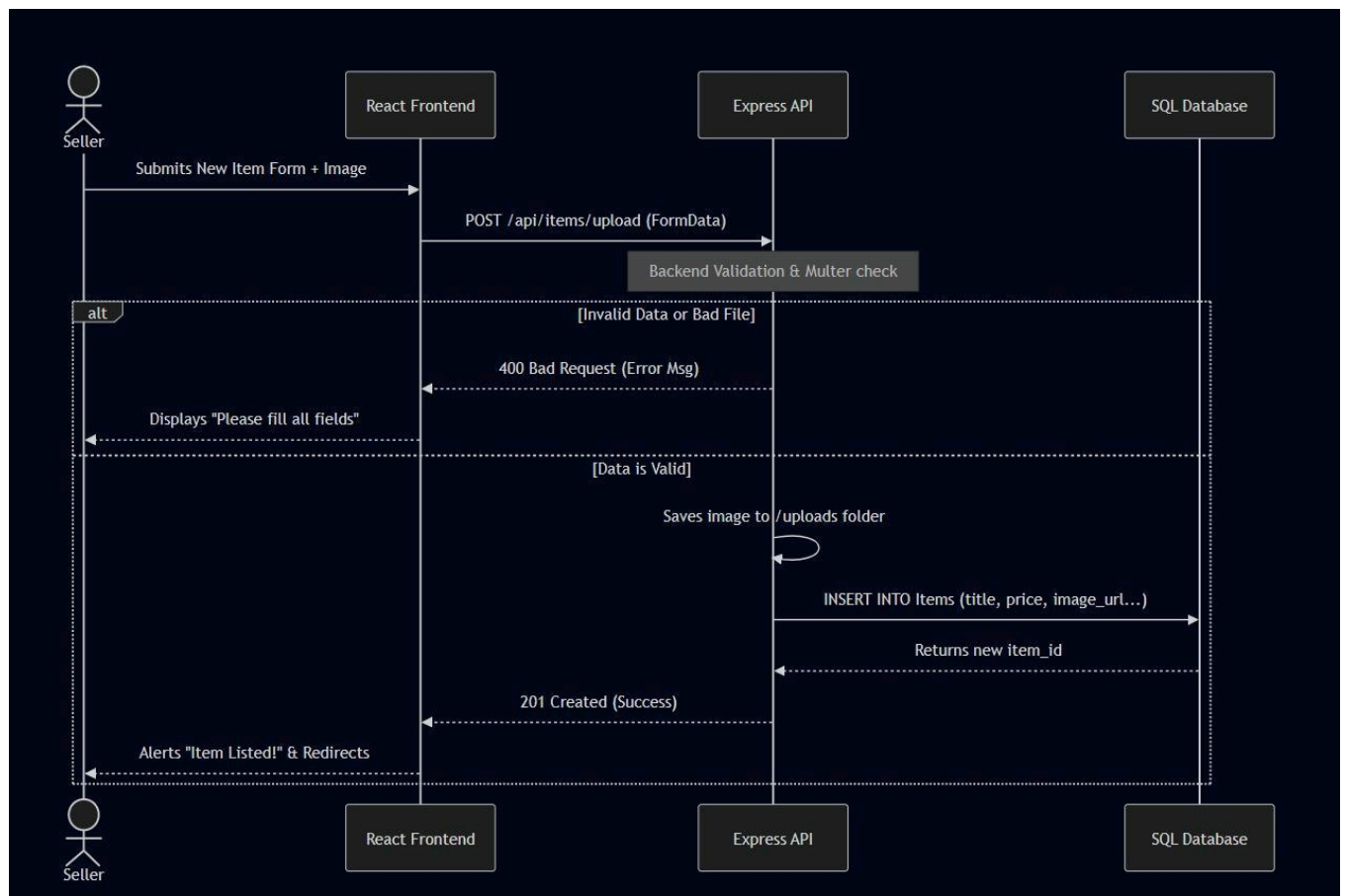


Figure 7: SD-02 — Add Listing Sequence Diagram (Seller → React → Express → SQL)

SD-03: Borrow Item Sequence

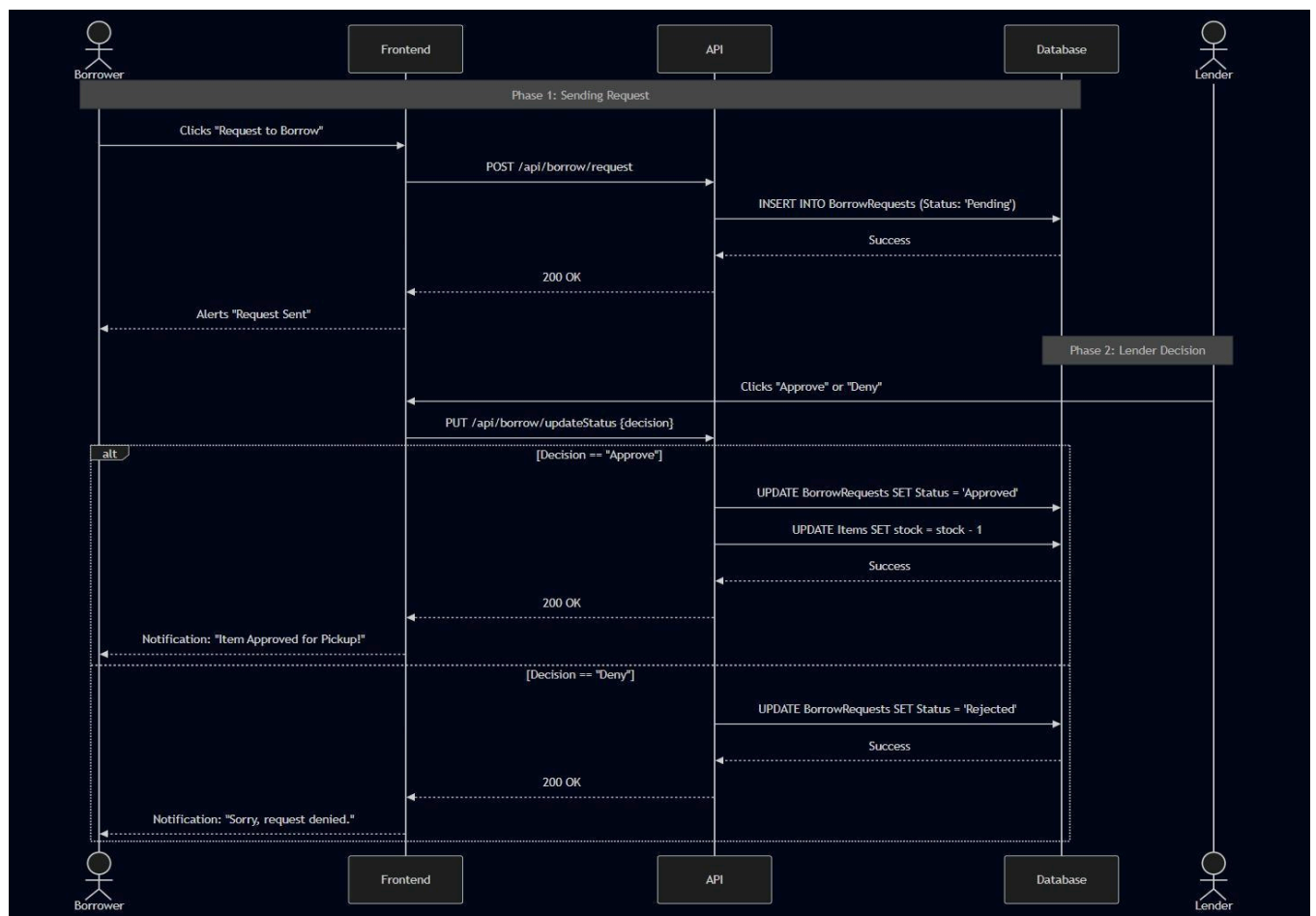


Figure 8: SD-03 — Borrow Item Sequence Diagram (Borrower → Frontend → API → Database → Lender)

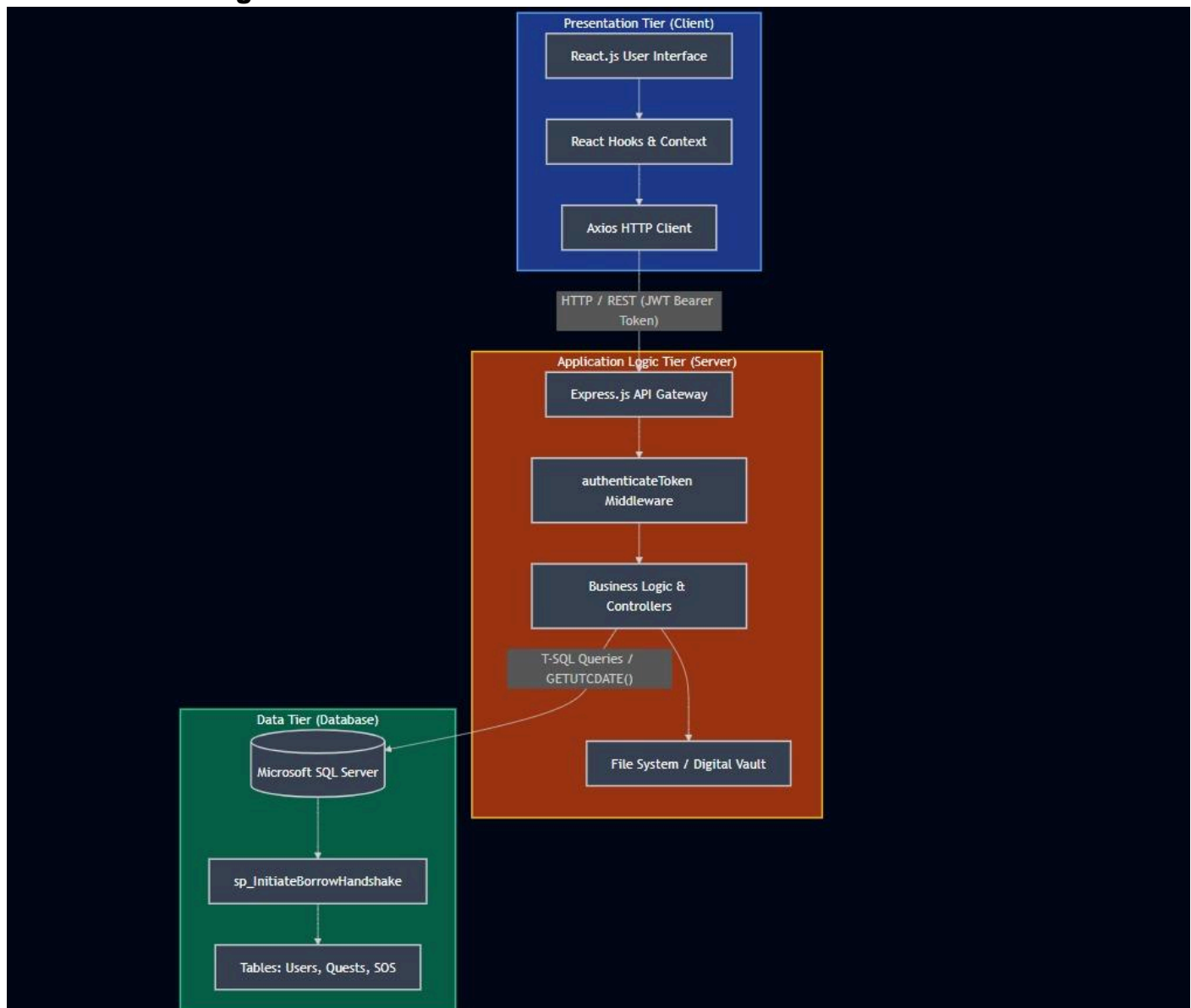
Architecture

UniThrift utilizes layered architecture, along with Microsoft SQL Server for robust relational data

constraints.

1. **Presentation Tier (Frontend):** Built with React.js and standard CSS. Uses React Hooks (useEffect, useCallback) for global state management and Axios for asynchronous HTTP communications.
2. **Logic Tier (Backend):** Built with Node.js and Express.js. Handles business logic, JWT-based security middleware (authenticateToken), file streaming for the Digital Vault, and RESTful routing.
3. **Data Tier (Database):** Hosted on Microsoft SQL Server. Enforces data integrity through Foreign Keys, CHECK constraints, mathematical stored procedures (sp_InitiateBorrowHandshake), and transactional logs using GETUTCDATE().

Architectural Diagram:



Implementation Screenshots

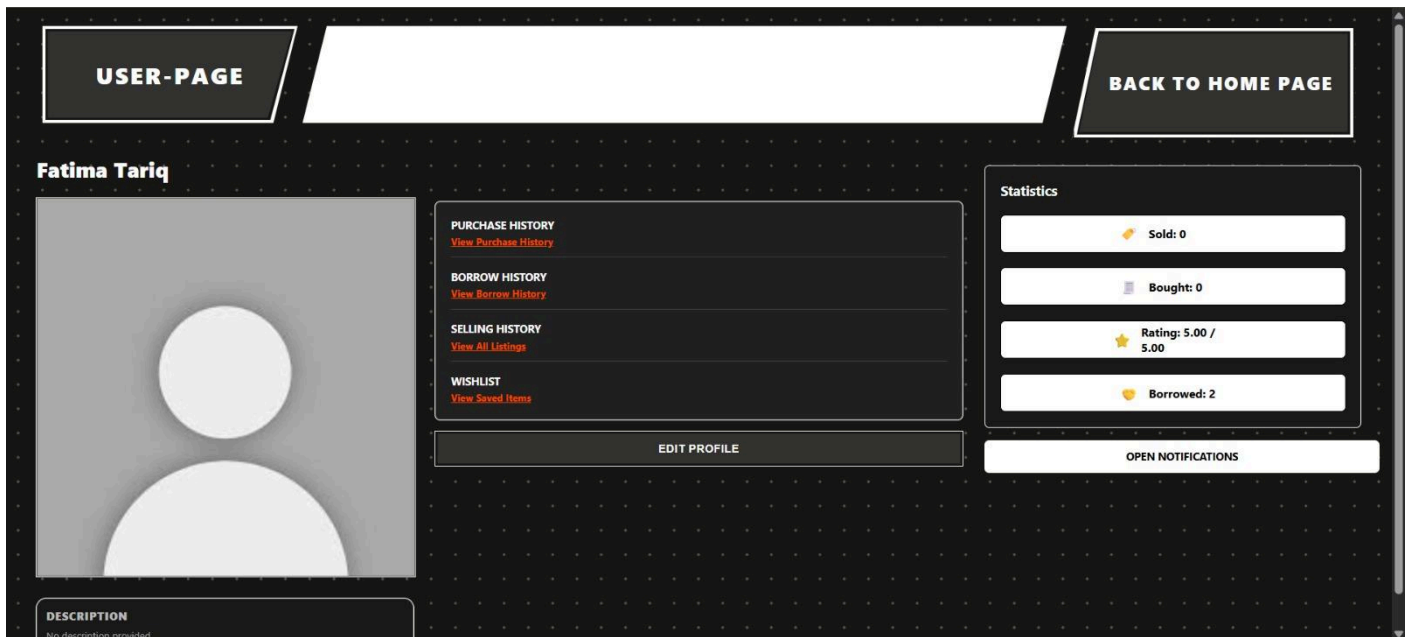
Frontend - University Email Auth & Profile Management:

The screenshot shows the 'UNI-THRIFT' login interface. It features a dark background with a subtle grid pattern. The login form is centered and contains the following elements:

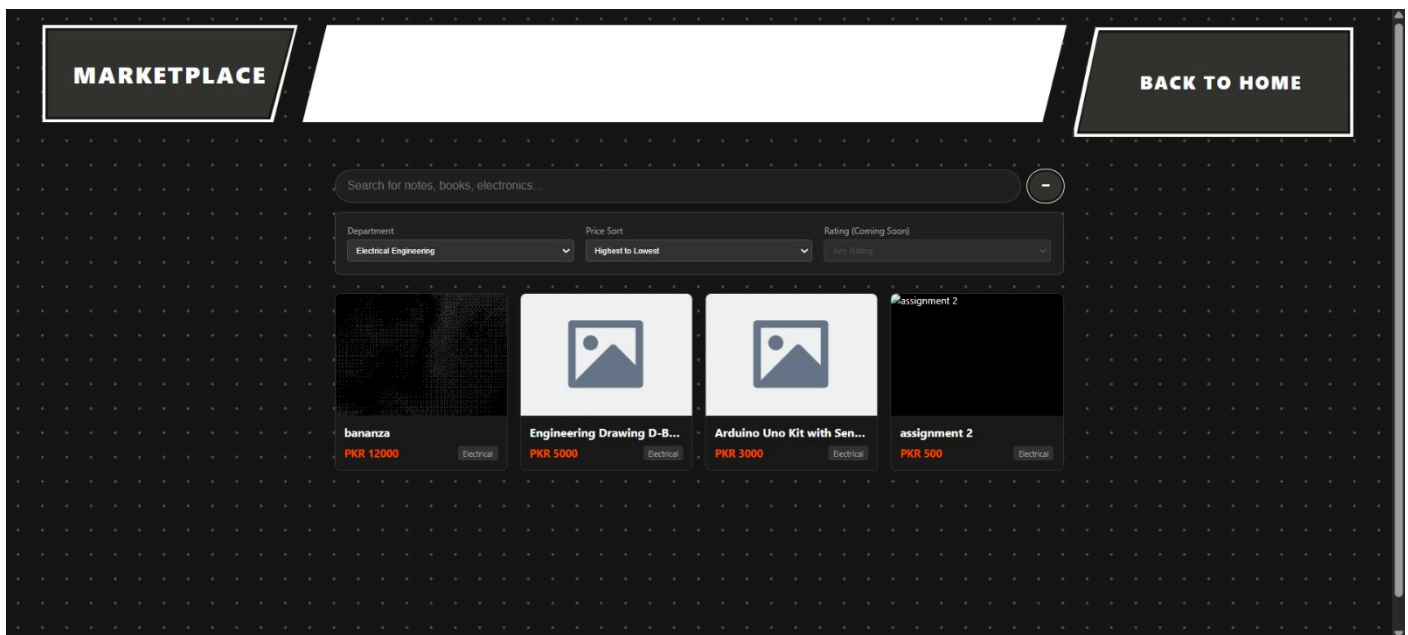
- UNI-THRIFT** (Header)
- LOGIN** (Section Header)
- UNIVERSITY EMAIL:** Input field containing 'i214567@thr.nu.edu.pk'
- PASSWORD:** Input field with masked characters '*****'
- GO** (Submit Button)
- No account? [Sign up here](#)

The screenshot shows the 'UNI-THRIFT' create account interface. It features a dark background with a subtle grid pattern. The create account form is centered and contains the following elements:

- UNI-THRIFT** (Header)
- CREATE ACCOUNT** (Section Header)
- FULL NAME:** Input field containing 'magbool'
- UNIVERSITY EMAIL:** Input field containing 'i1234@thr.nu.edu.pk'
- PASSWORD:** Input field with masked characters '*****'
- DEPARTMENT:** Dropdown menu showing 'Electrical Engineering'
- BIO (OPTIONAL):** Input field containing 'study and play' with a green checkmark icon
- PROFILE PICTURE:** Input field with a 'Choose File' button and the filename 'goat1.jpg'
- SIGN UP** (Submit Button)
- Already have an account? [Login here](#)



Frontend - Marketplace Discovery & Departmental Filtering:



Frontend - S.O.S Borrowing Requests & Real-Time Feed:

S.O.S REQUEST

BACK TO HOME

CALCULATOR

Requested by: Fatima Tariq

WILLING TO PAY

PKR 500

AMOUNT NEEDED

1

PRIORITY

HIGH

DETAILS & REQUIREMENTS

please i have exam

EDIT REQUEST

CANCEL REQUEST

UNI-THRIFT

Fatima

BUY

CREATE S.O.S BORROW REQUEST

Need something urgently? Broadcast a request to your department. Lenders will be notified immediately.

WHAT DO YOU NEED?

e.g. Engineering Drawing Kit

TARGET DEPARTMENT

Select Dept.

PRIORITY

High (Tomorrow)

WILLING TO PAY (PKR)

0

QUANTITY NEEDED

1

EXTRA DETAILS

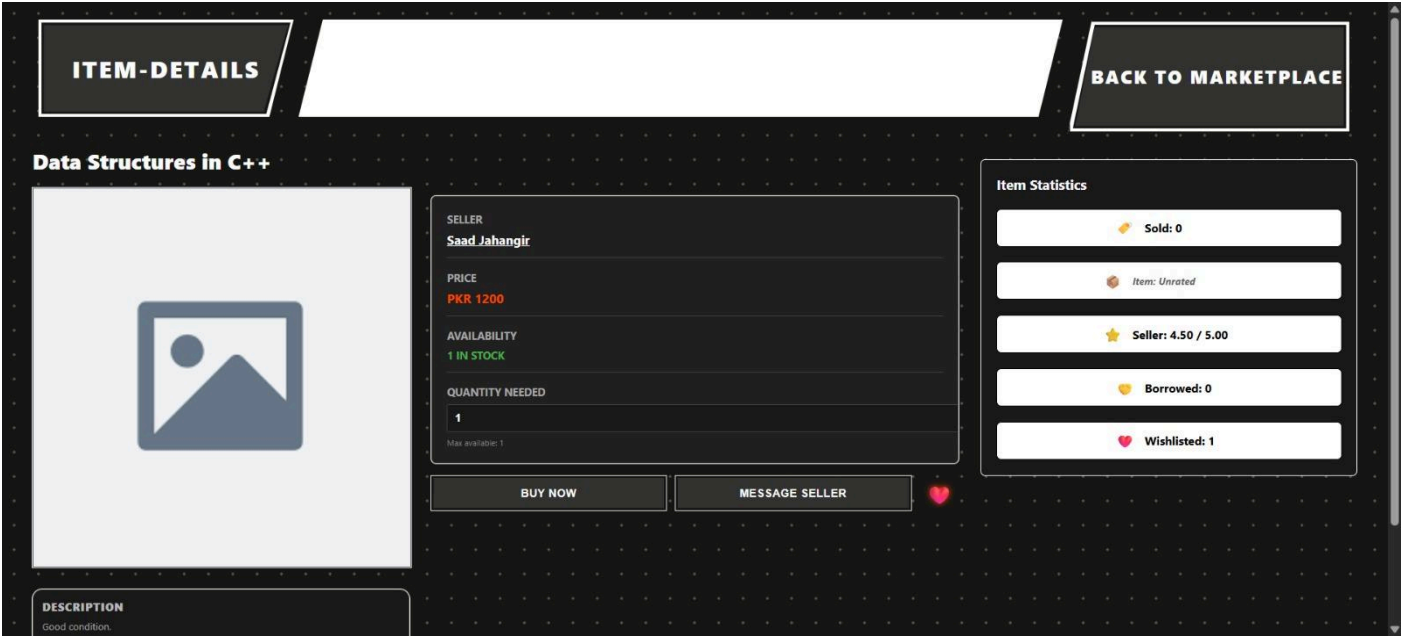
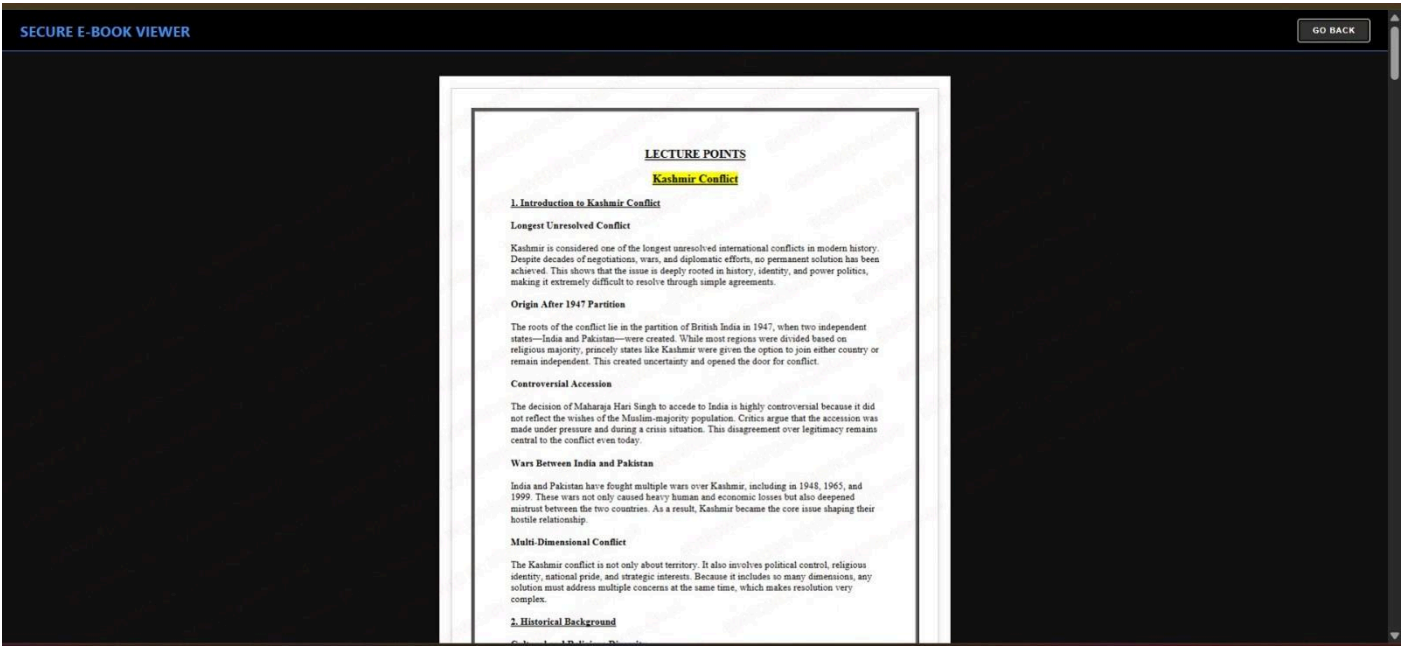
Specify condition needed, time you need it by, etc.

BROADCAST S.O.S

CANCEL

BORROW

Frontend - Secure Digital Vault & E-Book Bypass:



Frontend - In-App Messaging & Trust Rating System:

CHAT

BACK TO HOME

RECENT CONVERSATIONS



Saad Jahangir
hello brother

FROM: FATIMA TARIQ

TO: SAAD JAHANGIR

FATIMA TARIQ: hello brother

Write a message...

SEND

NOTIFICATION CENTER

READ ALL DMS

ACTIVE DIGITAL BORROWS



assignment 2

Expires: 5/16/2026

PENDING REVIEWS

Rate: assignment 2

Seller: Fatima Tariq

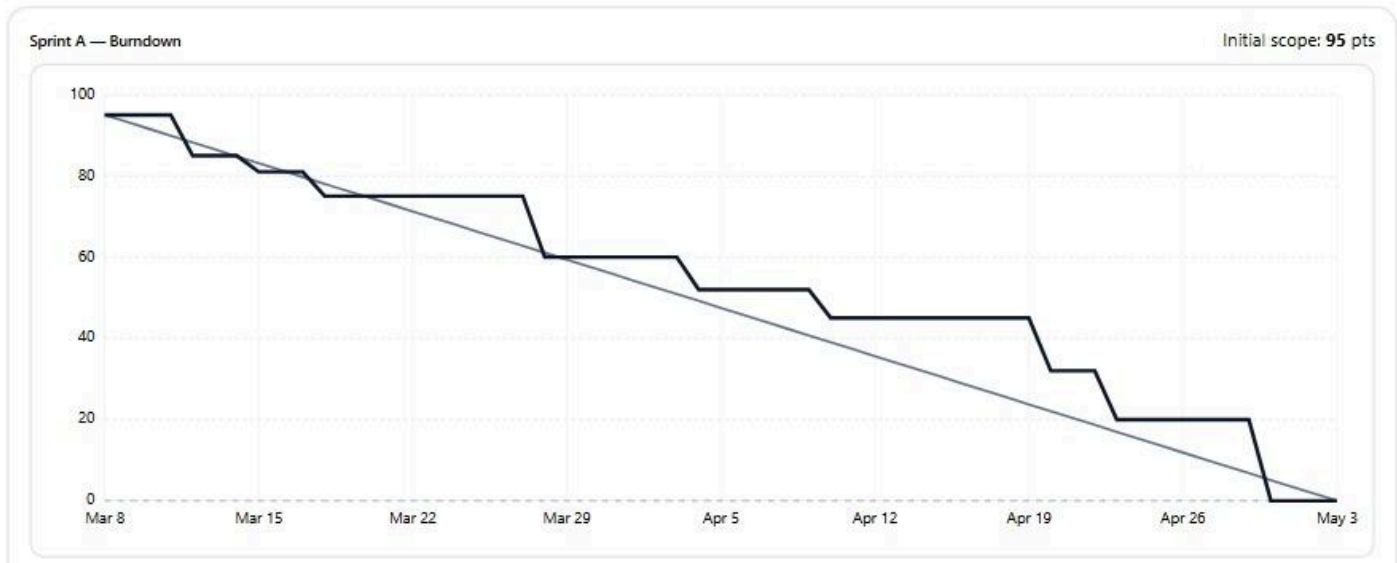


SUBMIT RATING

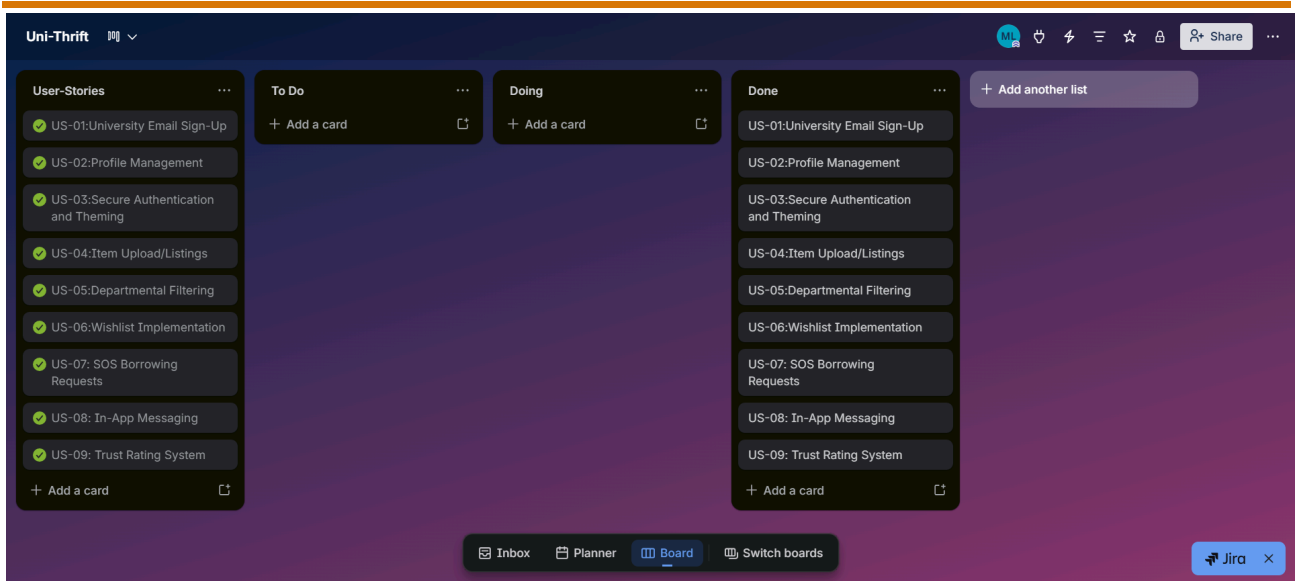
CLOSE

Product Burn-down Chart for the Project

Task	Points	Completed on
US-01 University Email Sign up	10	12/03/2026  Delete
US-02 Profile Management	06	18/03/2026  Delete
US-03 Secure Authentication	04	15/03/2026  Delete
US-04 Item Upload/Listing	15	28/03/2026  Delete
US-05 Departmental Filtering	08	04/04/2026  Delete
US-06 Wishlist Implimentation	07	10/04/2026  Delete
US-07 SOS Borrowing Requests	13	20/04/2026  Delete
US-08 In-App Messaging	20	30/04/2026  Delete
US-09 Trust Rating System	12	23/04/2026  Delete
Sum of task points: 95		



Trello Board Screen Shot



Boundary Value Analysis Testing

1. Login Interface

LOGIN INTERFACE BOUNDARY VALUE ANALYSIS

Testing boundaries for Email and Password inputs (evaluating strictly for empty vs. non-empty password fields)

TC	TEST CONDI- TION	INPU T: EMAI L	INPUT: PASS- WORD	EXPECTED RESULT	ACTUA L RESULT	ST A- TU S
01	Invalid Email + Valid Password	saad.unithrift	secure123	HTML5 prevents submission (invalid format)	UI blocks submission	Pass
02	Valid Email + Empty Password	saad@nu.edu.pk	(Empty)	System rejects auth due to missing password.	UI shows pass error	Pass
03	Valid Email + 1-Char Password (Min Valid)	a@nu.edu.pk	a	System accepts 1-char limit and attempts auth.	System attempts login	Pass
04	Max Valid Email + Max Valid Pass- word	longname@nu.e d	us.uppkerlongpa ss	The system handles extremely long boundaries safely.	System attempts login	Pass
05	Both Invalid Boundaries	admin@com	(Empty)	The system triggers multiple validation errors.	Form halted by UI	Pass

2. Registration Interface

REGISTRATION INTERFACE BOUNDARY VALUE ANALYSIS

Testing boundaries for Name, Domain-Restricted Email (nu.edu.pk), and Password (empty vs. non-empty).

TC	TEST CONDI- TION	NAME	EMAIL	PASS - WORD	EXPECTED RESULT	ACTUA L RESULT
01	All Valid Inputs (Happy Path)	Saad	s@nu.edu.pk	pass123	Validates all fields and registers users.	Regis- tration success - ful
02	Invalid Domain Email	Mahad	m@gmail.com	pass123	Backend specifically rejects non-nu.edu.pk domains.	Backend rejects
03	Empty Password	Soban	so@nu.edu.pk	(Empty)	Catches miss- ing password error and rejects.	UI shows error
04	Missing Depart- ment Dropdown	User	u@nu.edu.pk	pass123	HTML5 inter- cept triggers on an empty select field.	UI pre- vents submis- sion
05	Total Chaos (All Invalid)	(Empty)	admin.nu.edu.pk	(Empty)	Multiple validation triggers hit simultaneously.	Form halted by UI

Work Division Between Group Members

Muhammad Soban Sohail (Lead & Database Architect):

- Designed and structured the Microsoft SQL Server database schemas, including composite tracking structures for Transactions, Messages, and SOS_Requests.
- Engineered robust transaction logic to handle the complex mathematical state of physical handshakes, temporary stock locks, and inventory restocking upon rejected S.O.S. offers.
- Developed the complex SQL queries for the dynamic 5-star Trust Rating calculations (AVG) and User Profile statistical aggregations.
- Managed overall project direction, sprint tracking, and ERD generation.

Saad Jahangir (Full-Stack Developer):

- Built the entire Node.js/Express backend API and the responsive React.js frontend UI components.
- Engineered the client-side routing, Axios integrations, and the multer logic for securely uploading item images and PDFs to the server.
- Designed the authenticateToken JWT middleware and implemented the "Digital Vault Bypass" controller logic to differentiate instant E-book downloads from physical handshakes.
- Developed the real-time In-App Messaging UI, utilizing useCallback hooks for asynchronous message polling.

Mahad Jawad Rana (Requirement Engineer & QA Tester):

- Handled Requirement Engineering, translating complex e-commerce and borrowing logistics into concise Agile User Stories.
- Executed rigorous Boundary Value Analysis (BVA) testing on authentication routes and strict edge-case testing on the transaction API.
- Uncovered critical system bugs, including the massive GETDATE() vs GETUTCDATE() time-zone shift bug that was corrupting borrow expiration dates, and the exposed Wishlist JWT vulnerability.
- Managed the Trello board, generated the Burn-down charts, and compiled and formatted the final comprehensive project reports.

Lessons Learnt by Group

- **The Danger of Naive Timezones:** We learned that relying on the server's local clock (GETDATE()) causes massive data corruption when the frontend applies its own local PC timezone offset. Enforcing GETUTCDATE() universally on the backend was a major lesson in cross-platform data integrity, especially for strict borrow expirations.

- **Full-Stack Security Requires Alignment:** We discovered that simply adding JWT middle- ware to a backend API endpoint isn't enough. The frontend must be explicitly programmed to retrieve the token from sessionStorage and attach it to the Axios Authorization header, or the entire application breaks down with 401 Unauthorized errors.
- **Rigid Stored Procedures vs. Flexible APIs:** During Sprint 3, we realized that trying to force Digital E-Books through our strict physical handshake stored procedure caused database lockups. We learned that bypassing the procedure entirely at the Express API level for specific digital edge-cases is a much cleaner, more scalable architectural solution.
- **Asynchronous UX is crucial:** Transitioning our frontend to use asynchronous useEffect polling for the Chat and SOS Feed—rather than relying on full DOM refreshes—drastically improved the real-time, professional feel of the application.